EMAIL SYSTEM · V1.0

From RSVP to *Inbox*

The email system on the AI Salon Tel Aviv platform — from the moment a member clicks Register to the moment the 5-stage sequence lands in their inbox. Two parallel subsystems, one audit question: *was this email actually sent?*

Engineering | Operations | Mixed Audience | 2 Subsystems

Updated · 2026-07-07

DOC-EMAIL-001

01 · EXECUTIVE SUMMARY

Two Parallel Subsystems

The platform has **two independent email subsystems that do not share state**. The Legacy subsystem (`src/lib/email.ts` , Nodemailer/SMTP) handles transactional sends — RSVP confirmations, signup passwords, DM notifications, speaker messages — and writes **nothing** to the database. The Orchestrator subsystem (`src/lib/email-orchestrator/* + src/lib/email-campaign/*` , Gmail OAuth2 or mock) handles the 5-stage sequence, admin-built flows, and bulk campaigns — and writes **everything**: queue rows, sent snapshots, open/click tracking, reply matching. The central question this document answers is: **"How do I know a specific email was actually delivered?"** — and the answer depends entirely on which subsystem sent it.

Side-by-Side Comparison

LEGACY · SMTP

src/lib/email.ts

Nodemailer v9, SMTP transport. Fire-and-forget. Returns `{ ok, error? }`.

- RSVP confirmation (with .ics attachment)
- Signup / password reset
- DM notification (to recipient + cc ADMIN_EMAIL)
- Speaker-message relay (to ADMIN_EMAIL)

No DB log. Cannot answer "was this sent?" from the database.

ORCHESTRATOR · GMAIL

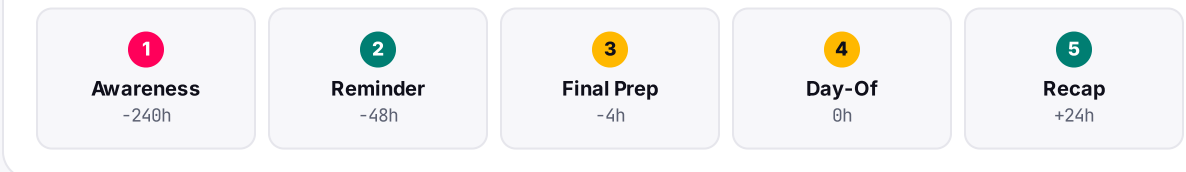
email-orchestrator/*

Gmail API v1 (OAuth2 refresh token) OR mock. Queue/worker model with full tracking.

- 5-stage sequence (Awareness → Recap)
- Admin-built EmailFlow steps (A/B test)
- Bulk campaigns via email-campaign/*
- Writes EmailQueue + TrackingLog + EmailRecipient + EmailEvent

Fully auditable. Per-recipient status, open pixel, click wrap, reply matching.

The 5-Stage Sequence Timeline



Templates, Tokens, Stop Rules

The 5-stage sequence is the platform's automated email nurture for event attendees. It is **seeded into the `EmailStageTemplate` table** by `src/lib/email-orchestrator/seed.ts` on every page load of `/admin/email` and `/admin/email/flows` (idempotent — safe to re-run). Each stage fires at a fixed offset relative to `Event.startsAt`, processed by the worker at `src/lib/email-orchestrator/worker.ts`. Two stages carry a "stop if not opened" rule: if the previous email was sent more than N hours ago and the recipient never opened it, all subsequent stages for that RSVP are marked SKIPPED. This prevents the platform from nagging disengaged members.

STAGE 1 Awareness

`-240h · stopIfNotOpened: 5h`

Sent **10 days before** the event. Subject: `"You're in! Here's what to expect at {{eventTitle}}"`. Body includes event title, formatted date, venue, address, speaker list, and a high-level agenda. This is the warmest touch — the member just registered, so engagement is highest here. If they don't open within 5 hours, Stage 2 will be SKIPPED.

TOKENS USED `{{firstName}}`, `{{eventTitle}}`, `{{eventDate}}`, `{{eventVenue}}`, `{{speakers}}`, `{{agenda}}`

STAGE 2 Reminder

`-48h · stopIfNotOpened: 24h`

Sent **48 hours before** the event. Subject: `"Reminder: {{eventTitle}} is in 48 hours"`. Lighter content — date, time, venue, agenda. If the recipient opened Stage 1 but doesn't open Stage 2 within 24 hours, Stages 3+4 still fire (the stop rule only checks the immediately-prior stage).

STAGE 3 Final Prep

`-4h · no stop rule`

Sent **4 hours before** the event. Subject: `"Final prep for {{eventTitle}} — see you soon"`. This is where the `{{checkInCode}}` token first appears — the email explicitly tells the recipient to show this code at the door. Doors-open timing is included.

VISUAL BUG If the user RSVP'd but never clicked "Check in" on the event page, `{{checkInCode}}` resolves to empty string. The template still prints the `"Check-in code:"` label, leaving a dangling label with no value.

STAGE 4 Day-Of

`0h (event start) · no stop rule`

Sent **at event start time**. Subject: `"Starting now: {{eventTitle}}"`. Includes the `{{checkInCode}}` token in monospace pink and the agenda. Same empty-token bug as Stage 3 if the user never self-checked-in.

STAGE 5 **Recap** +24h · no stop rule

Sent **24 hours after** the event. Subject: `"Thanks for coming to {{eventTitle}}"`. Includes links to event photos and recordings (when available). This stage fires regardless of door-checkin state — the worker's stop check only looks at `doorCheckedAt` for SKIPPING forward stages, not for halting the recap.

Template Tokens & Tracking Injection

The `renderTemplate()` function in `src/lib/email-orchestrator/templates.ts` performs simple `{{token}}` replacement against a context built from the RSVP, Event, and User rows. Supported tokens: `{{firstName}}`, `{{name}}`, `{{eventTitle}}`, `{{eventDate}}`, `{{eventVenue}}`, `{{eventAddress}}`, `{{eventUrl}}`, `{{checkInCode}}`, `{{speakers}}`, `{{agenda}}`. After token replacement, the renderer automatically injects an **open tracking pixel** (`<img src="{baseUrl}/api/track/email-open?id={queueId}">` — a 1×1 GIF appended before `</body>`) and **rewrites every** `href` to route through `/api/track/email-click?id={queueId}&target={encodeURIComponent(url)}` — skipping `mailto:`, `tel:`, and already-wrapped URLs. This is how the platform knows who opened and who clicked, without third-party tracking pixels.

03 · END-TO-END FLOW

RSVP → Email Orchestration

When a member clicks "Register to event" on `/e/[slug]`, the `POST /api/events/[slug]/rsvp` handler does two distinct email-related things: (1) it **synchronously sends the RSVP confirmation email** via the Legacy SMTP path (with a `.ics` calendar attachment), and (2) it **synchronously enqueues any matching flow steps** via `triggerFlowsForRsvp({triggerKind: "RSVP_GOING"})`. The 5-stage sequence is NOT triggered from this route — it's enqueued by the worker's bootstrap phase on the next worker tick. This split is the source of much confusion when debugging "why didn't the awareness email send?" — the answer is almost always "the worker hasn't run since the RSVP was created."

```
// RSVP ROUTE — EMAIL-RELATED CODE PATHS (SIMPLIFIED)
```

```
// 1. Synchronous RSVP confirmation email (Legacy SMTP, blocks response)
if (!wasAlreadyRegistered && emailConfigured()) {
  const ics = generateIcs(event);
  await sendRsvpConfirmationEmail({
    to: user.email, firstName, eventTitle, ...,
    attachments: [{ filename: "event.ics", content: ics }],
  });  // NOT logged to DB. Returns {ok, error?}.
}

// 2. Synchronous flow-trigger enqueue (does NOT send — just creates PENDING rows)
if (!wasAlreadyRegistered) {
  await triggerFlowsForRsvp({
    rsvpId: rsvp.id,
    triggerKind: "RSVP_GOING",
  });  // Creates EmailQueue rows for matching ACTIVE flow steps
}

// 3. The 5-stage sequence is NOT triggered here.
//    worker.ts::bootstrapStage1ForNewRsvps finds new RSVPs on each worker run.
```

Flow Builder Trigger Kinds & Where They Fire

RSVP_GOING
Fires from `/api/events/[slug]/rsvp` on first registration only.

DOOR_CHECKED_IN
Defined but never fires — see Known Issues.

MARKED_ATTENDED
Fires from `/api/admin/rsvps/[id]/attendance` on transition to ATTENDED.

MARKED_NO_SHOW
Fires from `/api/admin/rsvps/[id]/attendance` on transition to NO_SHOW.

MANUAL
Admin-triggered from the flow reports dashboard.

CRITICAL · WORKER NOT CRON-SCHEDULED

The orchestrator worker endpoint at `/api/email-orchestrator/run` is **not registered in `vercel.json`**. It only fires when an admin clicks "Run worker" on the Orchestrator admin tab. The 5-stage sequence + flow-builder steps will not fire automatically until either (a) an admin clicks the button, or (b) someone adds the route to `vercel.json` with a cron schedule. Vercel Hobby tier only allows daily crons.

04 · AUDIT QUERY COOKBOOK

"Was This Email Actually Sent?"

This is the central question. The answer depends entirely on which subsystem sent the email. Below are the exact Prisma queries for each subsystem — copy-paste these into `npx prisma studio` or your SQL client to verify delivery.

A. Legacy SMTP (RSVP confirmation, signup password, DM, speaker message)

AUDIT GAP · HIGH SEVERITY

There is **no query**. The DB writes nothing. `sendMail()` returns `{ ok, error? }` but writes no row. The only signal is server logs (`console.log` / `console.error` in the Vercel runtime logs). If a user reports "I never got my RSVP confirmation email", the only way to verify is to check Vercel logs for the request timestamp and look for the `[rsvp-confirmation]` log line or any SMTP error. There is no per-recipient delivery record. This is the single biggest audit gap in the email system.

B. Orchestrator 5-stage / Flow steps — query EmailQueue

The `EmailQueue` table (`prisma/schema.prisma`) holds one row per (RSVP × stage) or per (flow step × RSVP). The `status` field tells you exactly what happened.

STATUS	MEANING	HOW TO VERIFY
PENDING	Enqueued, waiting for `scheduledFor` to pass + worker tick	Check `scheduledFor` timestamp
SENT	Gmail API accepted the send. `sentAt` is set.	`sentAt IS NOT NULL`
OPENED	Recipient loaded the tracking pixel (1×1 GIF)	`openedAt IS NOT NULL`
CLICKED	Recipient clicked a wrapped link	`clickedAt IS NOT NULL`
SKIPPED	Stop rule fired (door checked, or prior stage not opened in time)	Check `errorMessage` for reason
FAILED	Gmail API rejected (quota, invalid address, etc.)	Check `errorMessage` + `attemptCount`

```
// PRISMA — WAS STAGE 3 (FINAL PREP) SENT TO THIS RSVP?
const q = await db.emailQueue.findMany({
  where: { rsvpId, stage: 3 },
  select: {
    status: true, sentAt: true, openedAt: true,
    subject: true, htmlBody: true,  // full rendered snapshot
    errorMessage: true, attemptCount: true,
  },
});
// status === "SENT" && sentAt !== null  →  actually sent.
// htmlBody contains the EXACT HTML that was sent — copy-paste to replay.
```

C. Bulk Campaigns — query EmailRecipient + EmailEvent

The campaign subsystem (`src/lib/email-campaign/*`) has its own audit trail: `EmailRecipient` (one row per campaign × email) for per-recipient state, and `EmailEvent` (one row per event) for the event stream.

```
// PRISMA — WAS THIS EMAIL SENT IN CAMPAIGN X TO USER Y?
```

```
const rec = await db.emailRecipient.findFirst({
  where: { campaignId, email: "eze@massapro.com" },
  include: { events: { orderBy: { createdAt: "asc" } } },
});
// rec.status === "SENT" && rec.sentAt !== null  →  actually sent.
// rec.openCount, rec.clickCount, rec.repliedAt  →  engagement.
// rec.messageId  →  SMTP Message-ID, used for IMAP reply matching.
// rec.events[]  →  full event stream: SENT, OPEN, CLICK, REPLY, BOUNCE, …
```

Worked Example: "Show me every email sent to eze@massapro.com in the last 7 days"

```
// THREE SEPARATE QUERIES — THERE IS NO UNIFIED EMAIL LOG
```

```
// 1. Orchestrator (5-stage + flow steps)
await db.emailQueue.findMany({
  where: { email: "eze@massapro.com", sentAt: { gte: new Date(Date.now() - 7*24*3600*1000) } },
});

// 2. Campaigns
await db.emailRecipient.findMany({
  where: { email: "eze@massapro.com", sentAt: { gte: new Date(Date.now() - 7*24*3600*1000) } },
});

// 3. Legacy SMTP (RSVP confirmations, signup, DMs, speaker messages)
//    → CANNOT BE QUERIED. Check Vercel runtime logs.
```

05 · ADMIN UI WALKTHROUGH

Where to Click in /admin/email

The email admin lives at `/admin/email` (server page: `src/app/admin/email/page.tsx`). It auto-seeds stage templates + the built-in Test audience on every load. The top-level nav (`EmailAdminNav` component) has three tabs, each backed by a different client component. Below is what each tab is for and the key actions on it.

CAMPAIGNS **Bulk Campaigns** `/admin/email (default tab)`

Use this when you want to send a one-off email blast to a list — a newsletter, an announcement, a sponsor spotlight. The Campaign Composer lets you pick a recipient list (all_members / non_members / event_rsvp / manual_upload / specific_users), write a subject + WYSIWYG HTML body, pick a From address, and either send immediately or schedule. After sending, the Campaign Stats view shows sent/failed/opened/clicked/replied/unsubscribed counts, a timeline chart, top-clicked-links map, recipient list (with CSV export), recent events stream, and a full email preview.

KEY BUTTONS — Refresh · Send due (triggers `/api/cron/email/send-scheduled`) · Poll replies (triggers `/api/cron/email/imap-poll`) · New campaign

ORCHESTRATOR

5-Stage Queue + Worker Control

`/admin/email?tab=orchestrator`

Use this when you want to inspect or debug the 5-stage sequence. Shows every `EmailQueue` row with filters by status, stage, event, and free-text search on email. Per-row actions: simulate open, simulate click (useful for testing the stop-rule logic without waiting for a real open). The top toolbar has the worker controls.

RUN WORKER	Triggers `/api/email-orchestrator/run` — runs both `runWorker()` (5-stage state machine) and `runFlowWorker()` (flow steps) in parallel. **This button MUST be clicked manually — there is no cron.**
PAUSE / RESUME	Toggles `emailSendPaused` SiteSetting. **Orchestrator only — does NOT pause Legacy SMTP.**

FLOWS

Flow Builder + Audiences + Templates

`/admin/email/flows`

Use this to build custom automated sequences beyond the 5-stage defaults. The Flow Builder canvas lets you chain up to 8 steps, each with: an audience (STATIC email list or DYNAMIC filter spec), a trigger kind (RSVP_GOING / DOOR_CHECKED_IN / MARKED_ATTENDED / MARKED_NO_SHOW / MANUAL), a template, an A/B subject variant test, and a delay (minutes/hours/days). Flow status: DRAFT / ACTIVE / PAUSED / ARCHIVED. Per-flow Report dialog shows per-step send counts + open rates with A/B variant breakdown.

SUB-TABS	Flows (canvas) · Audiences (STATIC + DYNAMIC lists, includes built-in Test audience) · Templates (EmailStageTemplate editor)

GAP · NO UNIFIED EMAIL LOG

There is **no single page** where you can see "every email sent across all subsystems in one list". The Orchestrator tab shows only `EmailQueue` rows (5-stage + flows). The Campaigns tab shows only `EmailRecipient` rows (bulk campaigns). Legacy SMTP sends (RSVP confirmation, signup password, DMs, speaker messages) are invisible to all admin UIs. To audit a Legacy SMTP send you have to dig through Vercel runtime logs.

06 · KNOWN ISSUES

Audit Gaps & Bugs to Fix

These are the gaps found during code review. Each is real, each affects either deliverability auditing or actual email delivery. Listed by severity. Fix recommendations are included where the fix is small.

HIGH · LEGACY SMTP NOT LOGGED

The Legacy SMTP path (`sendMail()` in `src/lib/email.ts`) writes nothing to the DB. RSVP confirmations, signup passwords, DM notifications, and speaker-message relays are completely invisible to the audit trail. You cannot answer "was the RSVP confirmation sent to user X on date Y?" from the database. **Fix:** add an `EmailLog` table (or extend `EmailQueue` with a `source` field) and write a row in `sendMail()` with `to` , `subject` , `status` , `error` , `sentAt` . This is the single biggest audit gap.

HIGH · WORKER NOT CRON-SCHEDULED

`/api/email-orchestrator/run` is missing from `vercel.json` . The 5-stage sequence + flow-builder steps only fire when an admin clicks "Run worker" manually. If nobody clicks the button for a day, no stages fire that day. **Fix:** add `{ "path": "/api/email-orchestrator/run", "schedule": "0 */1 * * *" }` to `vercel.json` (Vercel Hobby allows daily crons; Pro allows more frequent). Ensure `CRON_SECRET` is set in Vercel env and the route accepts it as `Authorization: Bearer <secret>` .

HIGH · DOOR_CHECKED_IN TRIGGER NEVER FIRES

The `DOOR_CHECKED_IN` trigger kind is defined in the flow-builder UI and the data model, but `/api/admin/check-in/confirm/route.ts` never calls `triggerFlowsForRsvp({triggerKind: "DOOR_CHECKED_IN"})` after setting `doorCheckedAt` . Any flow step configured with this trigger will silently never fire. **Fix:** after the `updateMany` succeeds (when `updateResult.count === 1` and the response status is `CONFIRMED`), call `triggerFlowsForRsvp({ rsvpId, triggerKind: "DOOR_CHECKED_IN" })` . Guard with `count === 1` so re-scans of an already-used code don't re-fire the trigger.

MEDIUM · PAUSE BUTTON ONLY PAUSES ORCHESTRATOR

The "Pause/Resume email sends" toggle on the Orchestrator admin tab sets the `emailSendPaused` SiteSetting, which is checked only in `src/lib/email-orchestrator/sender.ts` :: `sendEmail()` . The Legacy `sendMail()` function does NOT check this flag — RSVP confirmations, signup passwords, DMs, and speaker messages keep sending regardless. **Fix:** add the same pause check at the top of `sendMail()` in `src/lib/email.ts` , OR document clearly that the pause only applies to the orchestrator.

MEDIUM · CANCEL RSVP DOESN'T CANCEL PENDING EMAILS

The DELETE handler at `/api/events/[slug]/rsvp` removes the `EventRsvp` row but does NOT cancel pending `EmailQueue` rows for that RSVP. Subsequent stages may still fire to a deleted RSVP — the worker's stop-awareness only checks `doorCheckedAt` , not RSVP existence. **Fix:** in the DELETE handler, run `db.emailQueue.updateMany({ where: { rsvpId, status: "PENDING" }, data: { status: "SKIPPED", errorMessage: "RSVP cancelled" } })` before deleting the RSVP row.

LOW · EMPTY CHECK-IN CODE LABEL

Stages 3 and 4 hardcode the label `"Check-in code:"` even when `{{checkInCode}}` resolves to empty string (user RSVP'd but never clicked "Check in" on the event page). Recipients see a dangling label with no value. **Fix:** in `src/lib/email-orchestrator/templates.ts`, wrap the check-in-code block in a conditional that only renders if the token is non-empty. Either use a `{{#if checkInCode}}` handlebars-style directive (requires upgrading the renderer) or do a string-replace pass that strips the label if the token was empty.

07 · ENVIRONMENT VARIABLES

SMTP, Gmail, IMAP, Cron, Meta

All email-related env vars. Note that `.env.example` only documents the `SMTP_*` vars — the rest are only discoverable from source. **Production gotcha:** if `EMAIL_PROVIDER` is unset or set to anything other than `"gmail"`, the orchestrator silently runs in mock mode — it writes `EmailQueue` rows with `status=SENT` but no actual email goes out. Always verify `EMAIL_PROVIDER=gmail` in Vercel env before relying on orchestrator sends.

VARIABLE	REQ?	PURPOSE
SMTP_HOST	REQ	SMTP server hostname (e.g. smtp.gmail.com) — Legacy subsystem
SMTP_PORT	OPT	SMTP port (default 587)
SMTP_SECURE	OPT	"true" for SSL (465), else STARTTLS (587)
SMTP_USER	REQ	SMTP username
SMTP_PASS	REQ	SMTP password / app-specific password
SMTP_FROM	OPT	Default From for Legacy. Falls back to AI Salon Chat <chat@aisalon.massapro.com>
EMAIL_PROVIDER	REQ	"gmail" for real sends, anything else → mock (silent!). Orchestrator only.
EMAIL_SEND_ENABLED	OPT	"false" = hard kill switch for orchestrator (escapes even DB pause)
EMAIL_FROM	OPT	From for orchestrator sends. Default: AI Salon <noreply@aisalon.massapro.com>
GOOGLE_CLIENT_ID	REQ	Google OAuth2 client ID (shared with NextAuth)
GOOGLE_CLIENT_SECRET	REQ	Google OAuth2 client secret (shared with NextAuth)
GOOGLE_REFRESH_TOKEN	REQ	Offline refresh token for the sender Gmail account — get via the OAuth2 playground
IMAP_HOST	OPT	IMAP server for reply polling (campaign subsystem)
IMAP_USER	OPT	IMAP username — the inbox to scan for replies
IMAP_PASS	OPT	IMAP password
CRON_SECRET	REQ	Shared secret for /api/cron/* + /api/email-orchestrator/* routes. Checked as Authorization: Bearer <secret> or X-CRON-SECRET header.
ADMIN_EMAIL	OPT	Where DM notifications + speaker messages are CC'd/relayed. Default: eze@massapro.com
META_ACCESS_TOKEN	OPT	Meta Graph API token for Conversions API (email open/click → Meta CAPI)
META_PIXEL_ID	OPT	Meta Pixel ID — paired with access token

GOTCHA · INCONSISTENT FROM DEFAULTS

The default From address varies across 5 code paths: `SMTP_FROM` → `EMAIL_FROM` → `campaign.fromEmail` → `SMTP_USER` → hardcoded fallbacks like `chat@aisalon.massapro.com`, `noreply@aisalon.massapro.com`, `no-reply@aisalon.massapro.com`, `noreply@massapro.com`. **Production should set BOTH `SMTP_FROM` and `EMAIL_FROM` explicitly** so recipients see a consistent sender.

08 · VERIFICATION CHECKLIST

"I Just Sent a Campaign — How Do I Prove It Landed?"

Step-by-step ops checklist. Walk through this every time you send a campaign or want to verify a specific email was delivered. The numbered steps below assume you are signed in as an admin on `aisalon.massapro.com`.

1. **Before sending — verify env.** In Vercel project settings → Environment Variables, confirm `EMAIL_PROVIDER=gmail`, `GOOGLE_REFRESH_TOKEN` is set, and `EMAIL_SEND_ENABLED` is not `"false"`. If `EMAIL_PROVIDER` is missing or set to anything other than `"gmail"`, the orchestrator runs in mock mode — `EmailQueue` rows will be marked SENT but no actual email goes out.
2. **Before sending — verify pause is off.** Go to `/admin/email?tab=orchestrator` and confirm the "Pause email sends" button reads *Pause* (not *Resume*). If it reads Resume, the orchestrator is currently paused — click to resume.
3. **After sending — open the campaign.** Go to `/admin/email` (Campaigns tab). Click the campaign row. The Overview tab shows Sent / Failed / Opened / Clicked / Replied / Unsubscribed / Queued cards. A non-zero Sent count means the Gmail API accepted the send.
4. **Drill into a specific recipient.** Click the Recipients tab. Search by email. The status column shows per-recipient delivery state. `openCount` and `clickCount` confirm engagement. `repliedAt` confirms a reply was matched via IMAP polling.
5. **See the actual email that was sent.** Click the Email Preview tab. This shows the rendered HTML snapshot — the exact bytes that went out. If a recipient reports "I didn't get this", compare what they show you against this snapshot.
6. **For orchestrator stages — check the queue.** Go to `/admin/email?tab=orchestrator`. Filter by event. Each row shows status, scheduledFor, sentAt, openedAt. Click a row to see the rendered `htmlBody` snapshot. If status is `PENDING` past `scheduledFor`, the worker hasn't run — click "Run worker" at the top.
7. **Prove it's not demo data.** The built-in Test audience (3 hardcoded emails: `eze@massapro.com`, `ezeszna@gmail.com`, `eze@hi4.ai`) is the only "demo" artifact. Real campaign recipients come from the list-builder (all_members / event_rsvp / specific_users). The `EmailRecipient.email` field is the actual recipient email, not a placeholder. To confirm: filter the Recipients tab by any email — if you see real member emails with `status=SENT` and non-null `sentAt`, those are real sends.
8. **If something failed — check the event stream.** Click the Recent Events tab on the campaign. Each event has a `details` JSON with the failure reason. Common failures: Gmail quota exhausted, invalid email address, OAuth token expired (re-generate `GOOGLE_REFRESH_TOKEN`).
9. **If you cannot find an EmailQueue or EmailRecipient row — it was a Legacy SMTP send.** RSVP confirmations, signup passwords, DMs, and speaker messages go through `sendMail()` which writes nothing to the DB. Check Vercel runtime logs for the request timestamp and look for `[rsvp-confirmation]`, `[signup]`, `[dm]`, or `[speaker-message]` log lines.

Two subsystems, one *audit* question.

Legacy SMTP sends transactional emails with no DB log. The orchestrator + campaigns subsystem tracks everything — queue rows, sent snapshots, opens, clicks, replies. When asked "was this email sent?", first identify which subsystem sent it, then query the right table.

KEY TAKEAWAYS

- Legacy `sendMail()` writes **nothing** to the DB — Vercel logs only
- Orchestrator writes `EmailQueue` + `TrackingLog` — query by `(rsvpId, stage)`
- Campaigns write `EmailRecipient` + `EmailEvent` — query by `(campaignId, email)`
- `EMAIL_PROVIDER=gmail` required or orchestrator runs in mock mode
- Worker at `/api/email-orchestrator/run` is **not cron-scheduled** — click "Run worker" manually
- `DOOR_CHECKED_IN` trigger is defined but never fires — bug
- Pause button only pauses orchestrator, not Legacy SMTP